Имитационное моделирование

Лабораторная работа №5. Аппарат сетей Петри и задача обедающих философов

Ведьмина Александра Сергеевна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Подготовка проекта 8
4.2 Реализация модели сети Петри 9
4.3 Генерация literate-форматов 12
4.4 Базовый вычислительный сценарий 15
4.5 Параметрический сценарий 30
4.6 Выполнение Jupyter notebook 34
4.7 Автоматические тесты 35

5 Выводы **38**

Список литературы **39**

Список иллюстраций

4.1 Запуск Julia в каталоге `lab05` 8
4.2 Установка пакетов Julia для проекта `lab_05_models` 9
4.3 Запуск `generate.jl` и генерация производных форматов 12
4.4 Запуск `dining_philosophers_clean.jl` 21
4.5 Классическая сеть Петри для пяти философов 22
4.6 Сеть Петри с арбитром 23
4.7 Динамика маркировок в классической сети 25
4.8 Динамика маркировок в сети с арбитром 26
4.9 Сравнение стохастической и детерминированной динамики 28
4.10 Сравнение траекторий `Eat_i` для двух моделей 29
4.11 Запуск `dining_philosophers_param_clean.jl` 32
4.12 Вероятность и среднее время deadlock при изменении числа философов 33
4.13 Выполнение `dining_philosophers.ipynb` 34
4.14 Выполнение `dining_philosophers_param.ipynb` 34
4.15 Результат запуска `runtests.jl` 36

Список таблиц

4.1 Сводка базового стохастического прогона 24
4.2 Результаты параметрического исследования 32

1 Цель работы

Изучить аппарат сетей Петри на примере задачи обедающих философов, реализовать классическую и модифицированную модели на Julia, выполнить стохастическое и детерминированное моделирование, подготовить literate-источники, сгенерировать из них чистые скрипты, Jupyter notebook и Quarto-документацию, а затем интегрировать все полученные результаты в отчёт и презентацию.

2 Задание

В ходе лабораторной работы требовалось:

1. подготовить каталог проекта и зависимости Julia;
2. реализовать сеть Петри для задачи обедающих философов;
3. исследовать классическую модель и модификацию с арбитром;
4. обнаружить и проанализировать состояние `deadlock`;
5. перевести вычислительные сценарии в literate-стиль;
6. сгенерировать `.jl`, `.ipynb` и `.qmd`;
7. выполнить оба notebook;
8. провести дополнительное параметрическое исследование.

3 Теоретическое введение

Сеть Петри задаётся четвёркой

$$N = (P, T, F, M_0),$$

где `P` — множество позиций, `T` — множество переходов, `F` — множество дуг, а `M_0` — начальная маркировка [5; 6]. Переход может сработать только тогда, когда каждая входная позиция содержит необходимое число фишек.

Задача обедающих философов является классическим примером конкурентного доступа к разделяемым ресурсам [2]. В наивной постановке каждый философ берёт левую вилку, затем правую. Если все философы одновременно захватывают по одной вилке, возникает взаимная блокировка. Для устранения такого сценария вводится арбитр, ограничивающий число философов, которым разрешено одновременно входить в критическую секцию.

Стохастическое моделирование выполняется алгоритмом Гиллеспи [3]. Для воспроизводимости проекта используется `DrWatson.jl`, а для literate-кода — `Literate.jl` [1; 4; 7].

4 Выполнение лабораторной работы

4.1 Подготовка проекта

4.1.1 Запуск Julia

Работа начиналась с проверки установленной версии Julia и запуска REPL.

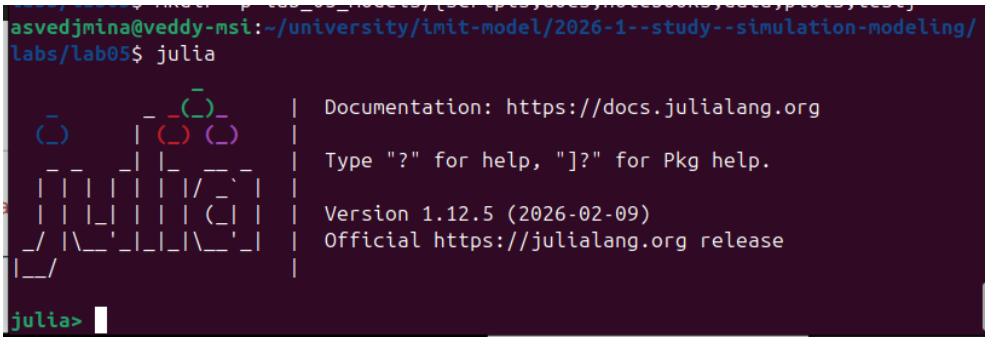


Рисунок 4.1: Запуск Julia в каталоге `lab05`

Версия `1.12.5` соответствует окружению, в котором затем запускались все скрипты лабораторной работы.

4.1.2 Установка зависимостей

После создания проекта были добавлены необходимые библиотеки: `CSV`, `DataFrames`, `DrWatson`, `IJulia`, `Literate`, `Plots`.

```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$ julia --project=lab_05_models -e 'using Pkg;
  Pkg.add(["CSV","DataFrames","DrWatson","IJulia","Literate","Plots"])'
   Resolving package versions...
      Compat entries added for CSV, DataFrames, DrWatson, IJulia, Literate, Plot
s
    Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b05/lab_05_models/Project.toml`
  [336ed68f] + CSV v0.10.16
  [a93c6f00] + DataFrames v1.8.2
  [634d3b9d] + DrWatson v2.19.1
  [7073ff75] + IJulia v1.34.4
  [98b081ad] + Literate v2.21.0
  [91a5bcdd] + Plots v1.41.6
    Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b05/lab_05_models/Manifest.toml`
  [66dad0bd] + AliasTables v1.1.3
  [d1d4a3ce] + BitFlags v0.1.9
  [336ed68f] + CSV v0.10.16
  [0b6fb165] + ChunkCodecCore v1.0.1
  [4c0bbee4] + ChunkCodecLibZlib v1.0.0
  [55437552] + ChunkCodecLibZstd v1.0.0
  [944b1d66] + CodecZlib v0.7.8
```

Рисунок 4.2: Установка пакетов Julia для проекта `lab_05_models`

В результате сформирован воспроизводимый проект `lab_05_models` со следующими каталогами:

- `src/` — исходный код модели;
- `scripts/` — literate-источники и clean-скрипты;
- `docs/` — Quarto-документы;
- `notebooks/` — Jupyter notebook;
- `data/` — таблицы с результатами;
- `plots/` — PNG-графики и GIF-анимация;
- `test/` — автоматические тесты.

4.2 Реализация модели сети Петри

Основной код сосредоточен в файле `src/DiningPhilosophers.jl`. В нём заданы:

- структура `PetriNet`;
- функции `build_classical_network` и `build_arbiter_network`;

- симуляторы `simulate_stochastic` и `simulate_deterministic`;
- функции `detect_deadlock`, `parameter_scan`, `plot_network`, `plot_marking_evolution`, `plot_parameter_summary`.

Ниже приведены ключевые фрагменты реализации.

4.2.1 Структура сети и дуги

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    pre::Matrix{Int}
    post::Matrix{Int}
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
    metadata::Dict{Symbol, Any}
end

function add_input_arc!(net::PetriNet, place::Int, transition::Int, weight::Int = 1)
    net.pre[place, transition] += weight
    net.incidence[place, transition] -= weight
    return net
end

function add_output_arc!(net::PetriNet, place::Int, transition::Int, weight::Int = 1)
    net.post[place, transition] += weight
    net.incidence[place, transition] += weight
    return net
end
```

4.2.2 Построение классической сети

```
function build_classical_network(N::Int)
    n_places = 4 * N
    n_transitions = 3 * N
    net = PetriNet(
        n_places,
        n_transitions;
        metadata = Dict(:N => N, :model => :classic, :has_arbiter => false),
    )
    ...
    return net, u0, copy(net.place_names)
end
```

4.2.3 Стохастический симулятор

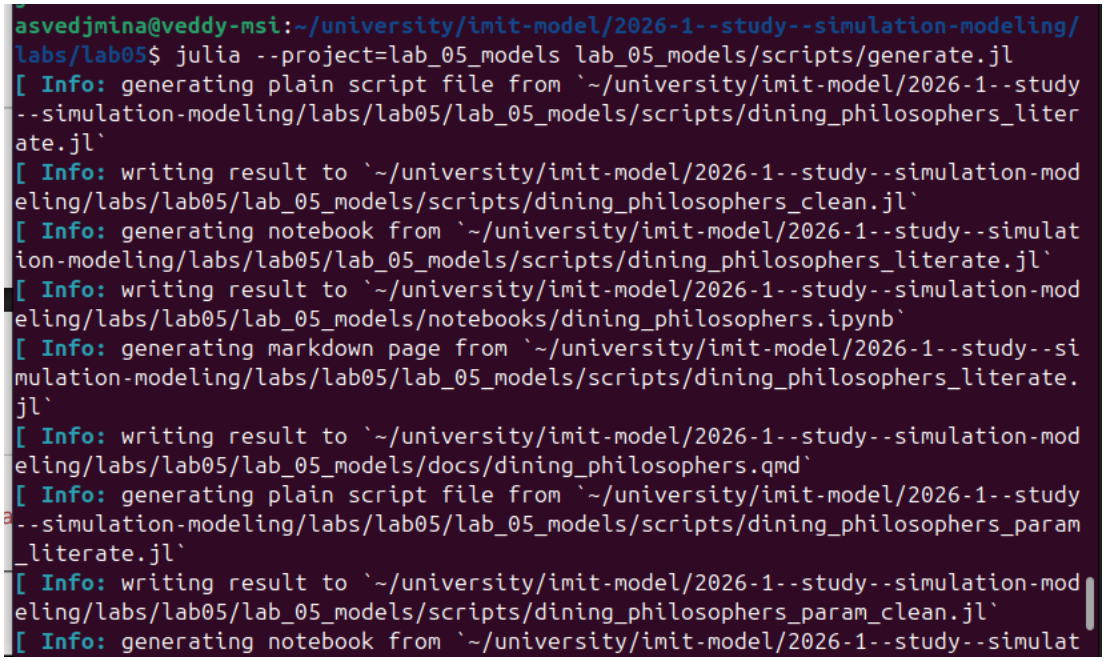
```
function simulate_stochastic(
    net::PetriNet,
    u0::AbstractVector{<:Real},
    tmax::Real;
    rates = ones(Float64, net.n_transitions),
    rng = Random.default_rng(),
    max_steps::Int = 200_000,
)
    marking = Float64.(u0)
    t = 0.0
    times = [t]
    states = [copy(marking)]
    ...
    return marking_dataframe(times, states, net.place_names)
```

```
end
```

Именно эта часть отвечает за имитацию событий и случайные интервалы времени между срабатываниями переходов.

4.3 Генерация literate-форматов

Переход от literate-источников к производным `.jl`, `.ipynb` и `.qmd` выполняется скриптом `scripts/generate.jl`.



```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$ julia --project=lab_05_models lab_05_models/scripts/generate.jl
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab05/lab_05_models/scripts/dining_philosophers_liter
ate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab05/lab_05_models/scripts/dining_philosophers_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulat
ion-modeling/labs/lab05/lab_05_models/scripts/dining_philosophers_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab05/lab_05_models/notebooks/dining_philosophers.ipynb`
[ Info: generating markdown page from `~/university/imit-model/2026-1--study--si
mulation-modeling/labs/lab05/lab_05_models/scripts/dining_philosophers_literate.
jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab05/lab_05_models/docs/dining_philosophers.qmd`
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab05/lab_05_models/scripts/dining_philosophers_param
_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab05/lab_05_models/scripts/dining_philosophers_param_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulat
```

Рисунок 4.3: Запуск `generate.jl` и генерация производных форматов

Выполнение этого шага создаёт:

- `scripts/dining_philosophers_clean.jl`;
- `scripts/dining_philosophers_param_clean.jl`;
- `notebooks/dining_philosophers.ipynb`;
- `notebooks/dining_philosophers_param.ipynb`;
- `docs/dining_philosophers.qmd`;

- docs/dining_philosophers_param.qmd.

4.3.1 Полный код scripts/generate.jl

```
using DrWatson
@quickactivate "lab_05_models"
using Literate

scripts_dir = @__DIR__
notebooks_dir = joinpath(@__DIR__, "..", "notebooks")
docs_dir = joinpath(@__DIR__, "..", "docs")

mkpath(notebooks_dir)
mkpath(docs_dir)

function quarto_header(title, description)
    return """
---
title: "$(title)"
description: "$(description)"
lang: ru-RU
jupyter: julia-1.12
format:
  html:
    toc: true
    number-sections: true
    code-fold: false
execute:
  cache: false
---
```

```
"""
end

sources = [
    (
        "dining_philosophers_literate.jl",
        "dining_philosophers",
        "Обедающие философы: базовые эксперименты",
        "Классическая сеть Петри, арбитр, графики",
    ),
    (
        "dining_philosophers_param_literate.jl",
        "dining_philosophers_param",
        "Параметрическое исследование",
        "Зависимость вероятности deadlock от числа философов",
    ),
]

for (src_file, base, title, description) in sources
    src_path = joinpath(scripts_dir, src_file)

    Literate.script(
        src_path,
        scripts_dir;
        name = base * "_clean",
        credit = false,
    )
    Literate.notebook(
        src_path,
```

```
        notebooks_dir;
        name = base,
        execute = false,
        credit = false,
    )
    Literate.markdown(
        src_path,
        docs_dir;
        name = base,
        flavor = Literate.QuartoFlavor(),
        credit = false,
    )

    qmd_path = joinpath(docs_dir, base * ".qmd")
    content = read(qmd_path, String)
    write(qmd_path, quarto_header(title, description) * content)
end

println("Literate generation completed.")
```

4.4 Базовый вычислительный сценарий

4.4.1 Полный код `scripts/dining_philosophers_literate.jl`

```
# # Обедающие философы: базовые эксперименты
#
# Этот literate-скрипт воспроизводит основную часть лабораторной работы 5.
# Он строит классическую сеть Петри и её модификацию с арбитром,
# выполняет стохастическое и детерминированное моделирование
# и сохраняет траектории, графики и GIF-анимацию.
```

```
#
# ## Подключение библиотек

using DrWatson
@quickactivate "lab_05_models"

ENV["GKSwstype"] = "100"

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers
using CSV
using DataFrames
using Plots
using Random

mkpath(datadir())
mkpath(plotsdir())

# ## Базовые параметры
#
# Используется стандартная постановка из пяти философов
# и горизонт моделирования `50` единиц времени.

N = 5
tmax = 50.0
classic_seed = 2026
arbiter_seed = 2027

# ## Сети Петри
#
```

```
# Сначала формируются две сети:
# классическая и модифицированная с арбитром.

net_classic, u0_classic, _ = build_classical_network(N)
net_arbiter, u0_arbiter, _ = build_arbiter_network(N)

classic_network_plot = plot_network(net_classic)
arbiter_network_plot = plot_network(net_arbiter)

classic_network_plot
arbiter_network_plot

#jl savefig(classic_network_plot, plotsdir("classic_network.png"))
#jl savefig(arbiter_network_plot, plotsdir("arbiter_network.png"))

# ## Стохастическое моделирование
#
# Для каждой сети выполняется симуляция по алгоритму Гиллеспи.

df_classic = simulate_stochastic(
    net_classic,
    u0_classic,
    tmax;
    rng = MersenneTwister(classic_seed),
)
df_arbiter = simulate_stochastic(
    net_arbiter,
    u0_arbiter,
    tmax;
    rng = MersenneTwister(arbiter_seed),
```

```
)

CSV.write(datadir("dining_classic.csv"), df_classic)
CSV.write(datadir("dining_arbiter.csv"), df_arbiter)

deadlock_classic = detect_deadlock(df_classic, net_classic)
deadlock_arbiter = detect_deadlock(df_arbiter, net_arbiter)

deadlock_summary = DataFrame(
    model = ["classic", "arbiter"],
    deadlock = [deadlock_classic, deadlock_arbiter],
    deadlock_time = [
        deadlock_time(df_classic, net_classic),
        deadlock_time(df_arbiter, net_arbiter),
    ],
    final_total_eaters = [
        total_eaters(df_classic, N)[end],
        total_eaters(df_arbiter, N)[end],
    ],
)
deadlock_summary

#jl CSV.write(datadir("deadlock_summary.csv"), deadlock_summary)

# ## Графики маркировки
#
# Динамика по позициям позволяет визуально увидеть,
# что в классической сети система приходит к тупику,
# а с арбитром остаётся активной.
```

```
classic_marking_plot = plot_marking_evolution(df_classic, net_classic)
arbiter_marking_plot = plot_marking_evolution(df_arbiter, net_arbiter)

classic_marking_plot
arbiter_marking_plot

#jl savefig(classic_marking_plot, plotsdir("classic_simulation.png"))
#jl savefig(arbiter_marking_plot, plotsdir("arbiter_simulation.png"))

# ## Детерминированная аппроксимация
#
# Для сравнения строится
# простая mean-field аппроксимация по тем же сетям.

df_classic_det = simulate_deterministic(net_classic, u0_classic, tmax)
df_arbiter_det = simulate_deterministic(net_arbiter, u0_arbiter, tmax)

deterministic_plot = plot_total_eaters_comparison(
    df_classic,
    df_classic_det,
    df_arbiter,
    df_arbiter_det,
    N,
)
deterministic_plot

#jl CSV.write(datadir("dining_classic_deterministic.csv"), df_classic_det)
#jl CSV.write(datadir("dining_arbiter_deterministic.csv"), df_arbiter_det)
#jl savefig(deterministic_plot, plotsdir("deterministic_comparison.png"))
```

```
# ## Сравнение по состояниям `Eat_i`
#
# Итоговый график сопоставляет количество философов
# в состоянии приёма пищи для двух вариантов сети.

final_report_plot = plot_eating_comparison(df_classic, df_arbiter, N)
final_report_plot

#jl savefig(final_report_plot, plotsdir("final_report.png"))

# ## Анимация
#
# Для наглядности дополнительно строится GIF-анимация
# для классической сети при `N = 3`.

net_anim, u0_anim, _ = build_classical_network(3)
df_anim = simulate_stochastic(net_anim, u0_anim, 30.0; rng = MersenneTwister(123))

#jl animate_marking(
#jl     df_anim,
#jl     net_anim,
#jl     plotsdir("philosophers_simulation.gif");
#jl     fps = 2,
#jl )

# ## Краткие выводы
#
# 1. В классической сети при выбранном `seed` возникает deadlock.
# 2. В модификации с арбитром deadlock не возникает.
# 3. Детерминированная аппроксимация подтверждает,
```

```
#    что арбитр поддерживает ненулевой поток
#    переходов в состояниях `Eat_i`.
```

4.4.2 Запуск clean-скрипта

Основной сценарий был выполнен из clean-версии literate-файла.

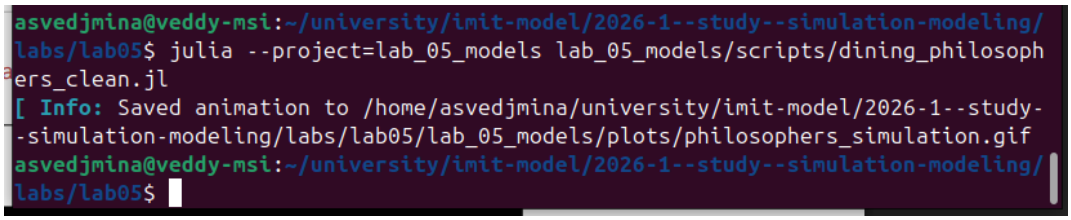


Рисунок 4.4: Запуск `dining_philosophers_clean.jl`

В ходе этого запуска были созданы:

- `dining_classic.csv`;
- `dining_arbiter.csv`;
- `dining_classic_deterministic.csv`;
- `dining_arbiter_deterministic.csv`;
- `deadlock_summary.csv`;
- `classic_network.png`, `arbiter_network.png`;
- `classic_simulation.png`, `arbiter_simulation.png`;
- `deterministic_comparison.png`, `final_report.png`;
- `philosophers_simulation.gif`.

4.4.3 Схемы сетей Петри

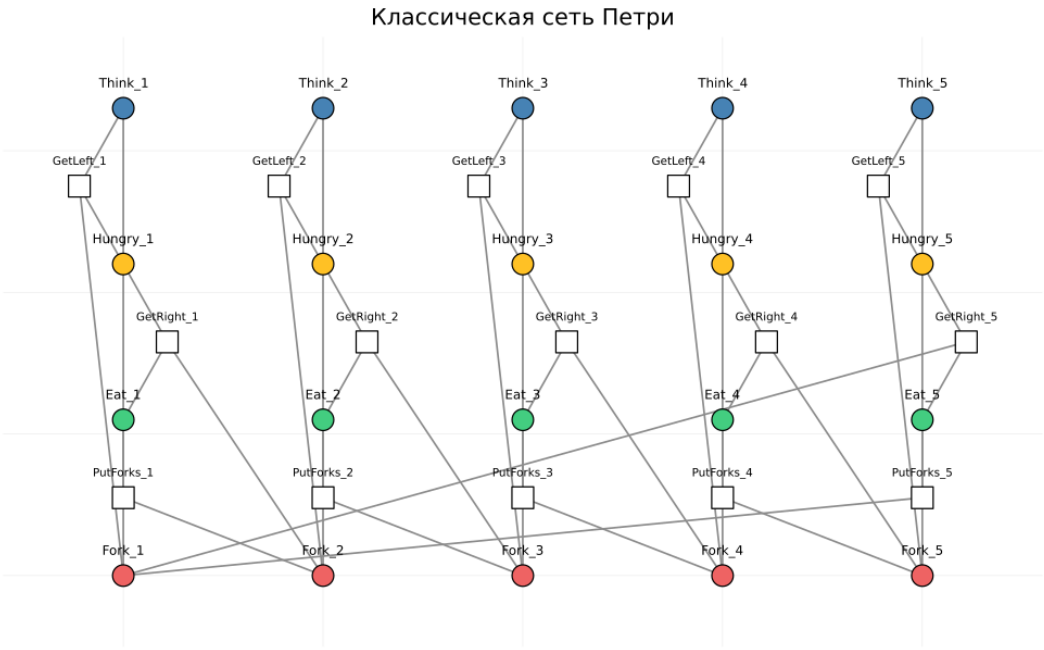


Рисунок 4.5: Классическая сеть Петри для пяти философов

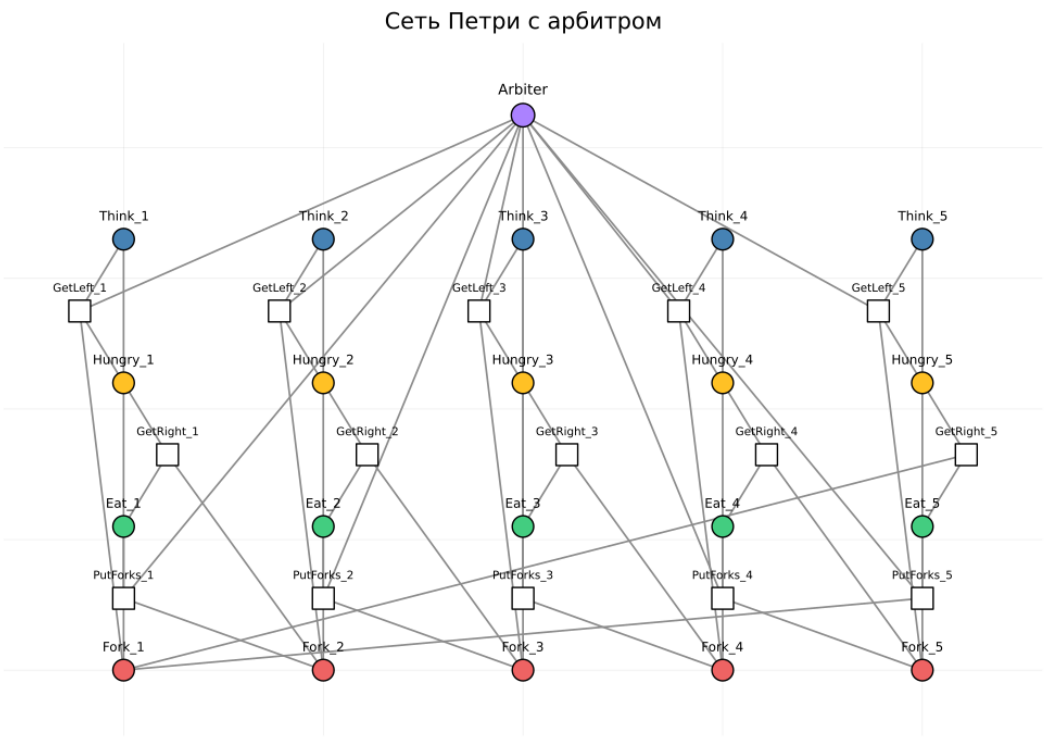


Рисунок 4.6: Сеть Петри с арбитром

В классической сети присутствует сценарий, при котором все философы блокируют друг друга. В модифицированной сети с арбитром дополнительная позиция ограничивает число одновременно активных философов до `N-1`.

На схеме классической сети видно, что каждая пара соседних философов делит одну и ту же вилку, а захват ресурса разбит на два последовательных перехода: сначала `GetLeft_i`, затем `GetRight_i`. Именно эта последовательность создаёт возможность циклического ожидания. На схеме сети с арбитром появляется дополнительная позиция `Arbiter`, из которой изымается одна фишка при входе в критическую секцию и возвращается после освобождения вилок. Тем самым в системе исключается состояние, в котором все философы одновременно удерживают по одной вилке.

4.4.4 Численные результаты базового прогона

Итог одного базового стохастического запуска приведён в табл. 4.1.

Таблица 4.1: Сводка базового стохастического прогона

Модель	Наличие deadlock	Время наступления	Итоговое число философов в `Eat`
Классическая сеть	`true`	`0.6297`	`0.0`
Сеть с арбитром	`false`	—	`1.0`

4.4.5 Эволюция маркировок

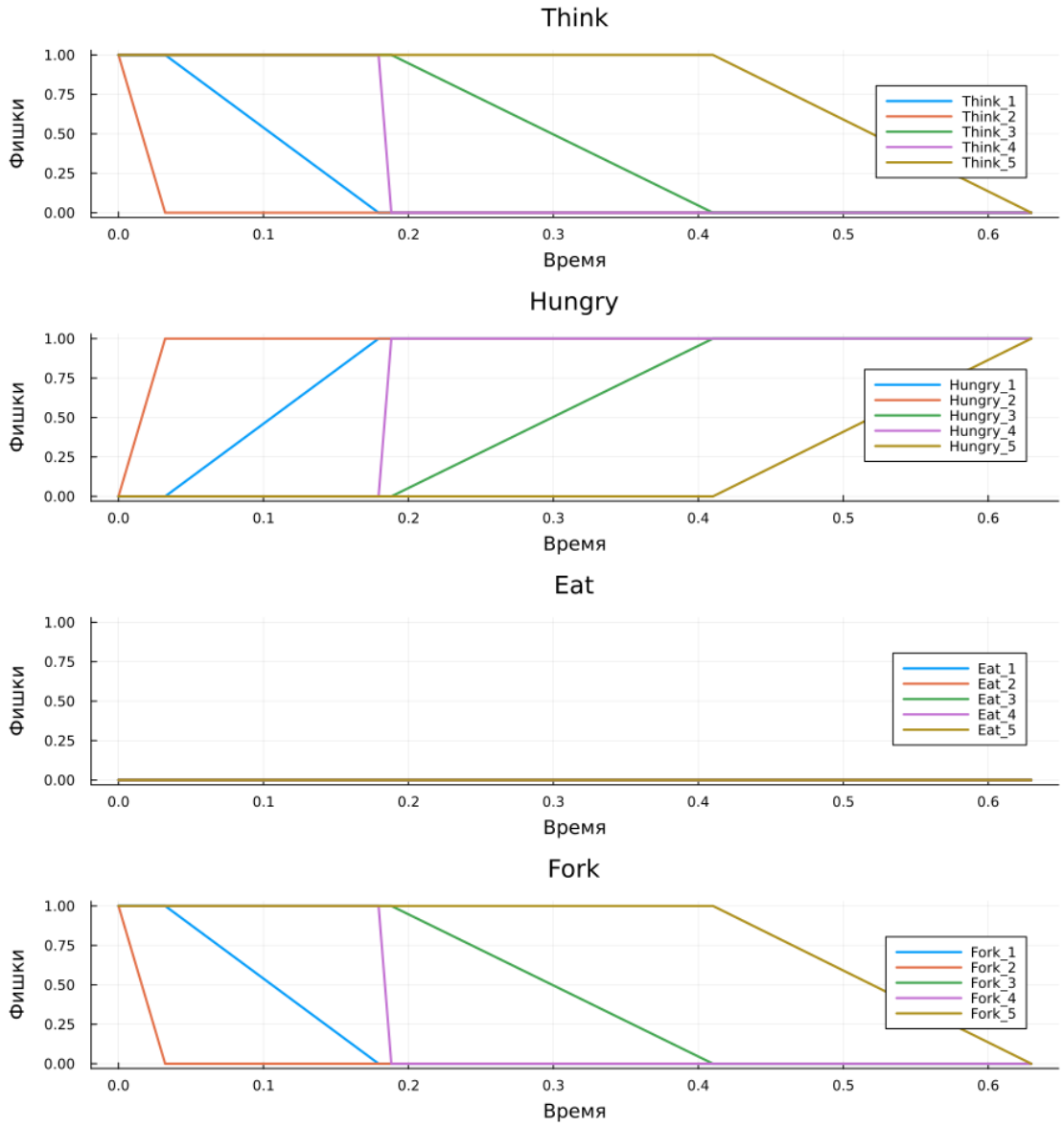


Рисунок 4.7: Динамика маркировок в классической сети

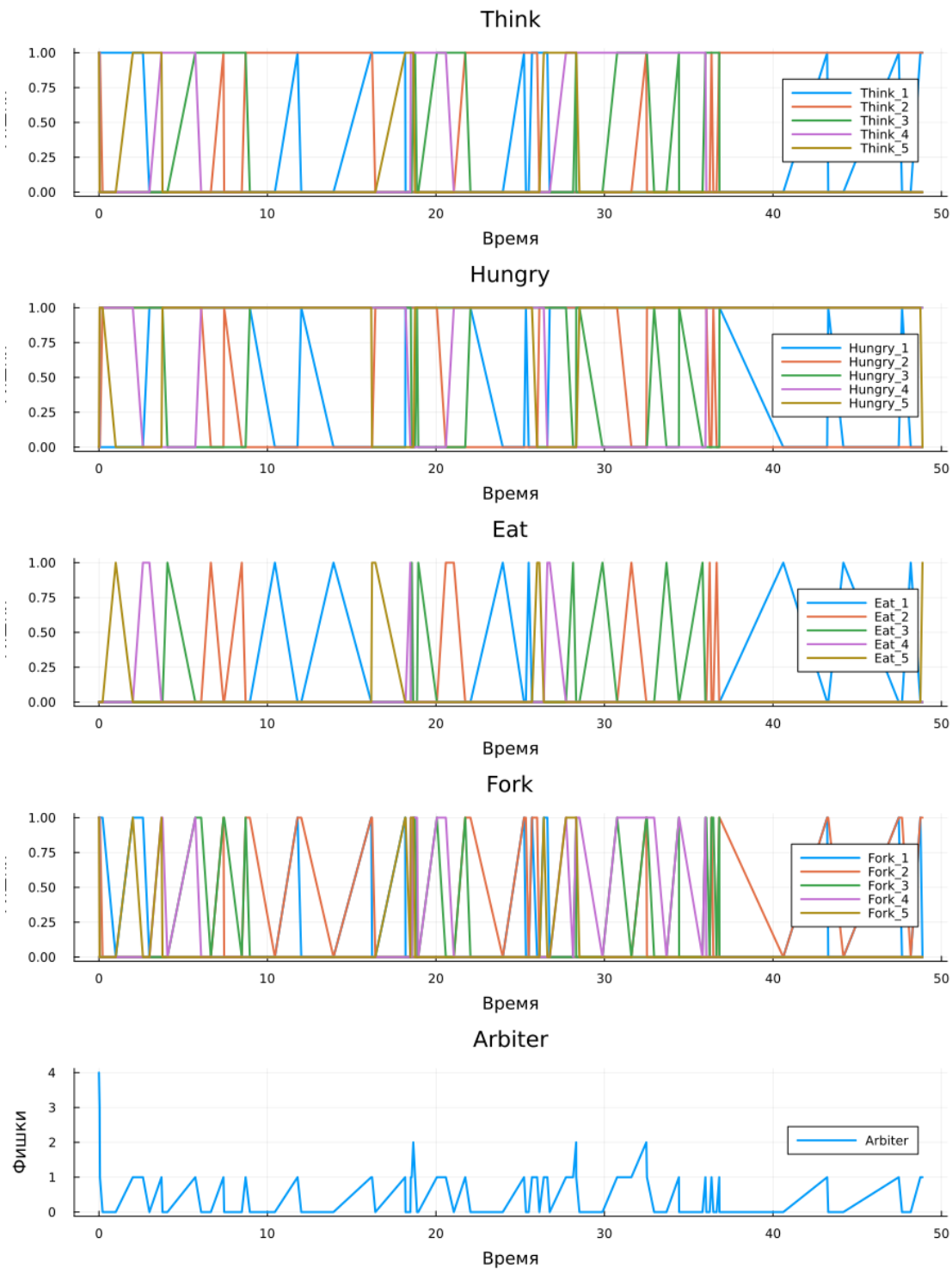


Рисунок 4.8: Динамика маркировок в сети с арбитром

На рис. 4.7 видно быстрое вырождение системы в тупиковое состояние: все философы переходят в `Hungry_i`, а срабатывания переходов прекращаются. На рис. 4.8 сеть с арбитром остаётся активной: состояния `Think`, `Hungry` и `Eat` продолжают чередоваться.

Если рассматривать панели подробнее, то в классической сети маркеры в позициях `Fork_i` быстро исчезают из свободных вилок и перестают возвращаться назад, а линии `Eat_i` после короткого всплеска обнуляются и уже не восстанавливаются. Одновременно линии `Hungry_i` стабилизируются на ненулевом уровне, что и соответствует взаимной блокировке. В сети с арбитром такой стабилизации нет: освобождение вилок происходит регулярно, линии `Fork_i` колеблются, а состояния `Eat_i` периодически активируются у разных философов. Это означает, что сеть остаётся живой и переходы продолжают срабатывать на всём интервале моделирования.

4.4.6 Сравнение стохастической и детерминированной моделей

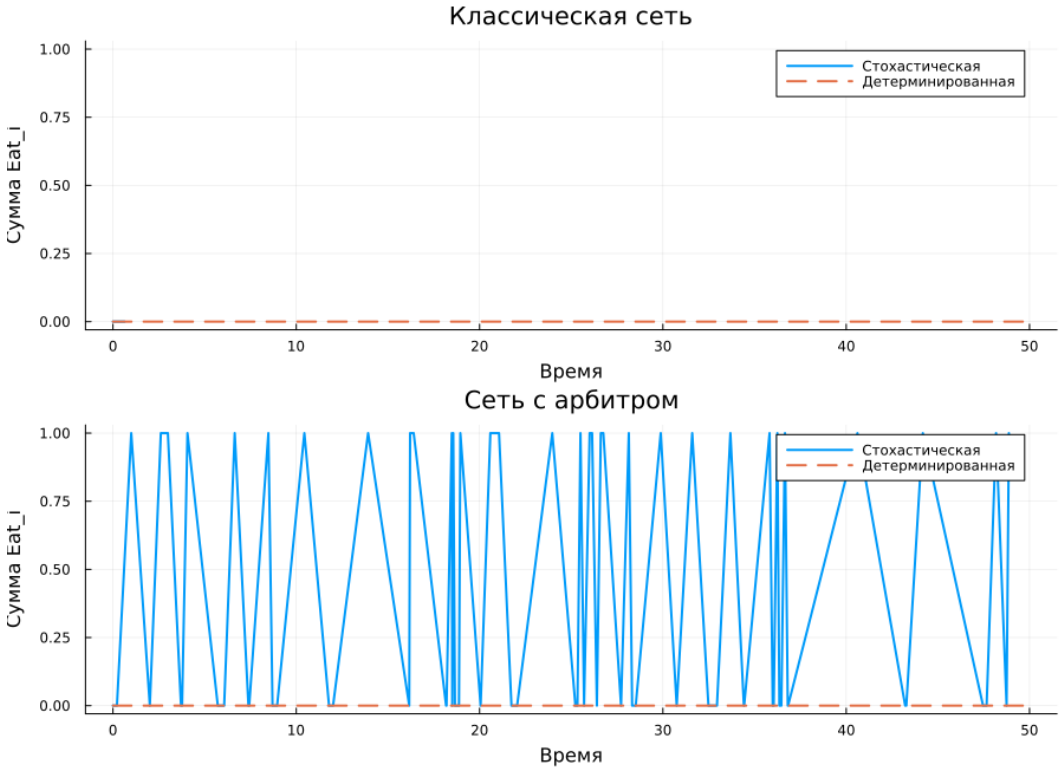


Рисунок 4.9: Сравнение стохастической и детерминированной динамики

Детерминированная аппроксимация подтверждает качественный вывод: классическая постановка теряет активность, а вариант с арбитром сохраняет ненулевой поток переходов в состояниях `Eat_i`.

На верхней панели сравнения видно, что стохастическая траектория классической сети быстро падает к нулю и далее совпадает с детерминированной тенденцией затухания. На нижней панели для сети с арбитром обе кривые остаются выше нуля: число философов, находящихся в состоянии `Eat`, колеблется, но не замирает. Это важно, потому что детерминированная модель здесь не просто повторяет форму одной случайной траектории, а воспроизводит общий режим функционирования сети.

4.4.7 Итоговый сравнительный график

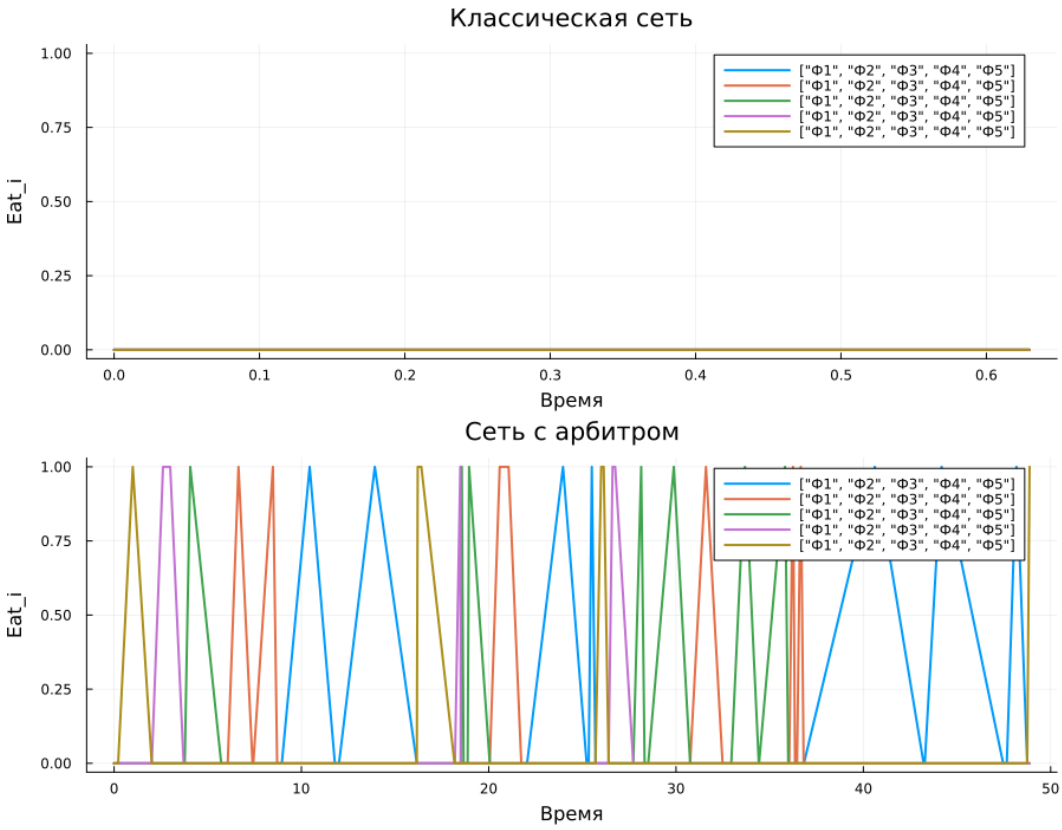


Рисунок 4.10: Сравнение траекторий `Eat_i` для двух моделей

Этот график служит наглядным подтверждением того, что после введения арбитра философы продолжают получать доступ к ресурсам, тогда как в классической сети все траектории `Eat_i` в итоге обнуляются.

В классической сети все линии `Eat_i` достаточно быстро приходят к нулю и далее лежат на оси, что означает полное прекращение приёма пищи всеми философами. Для сети с арбитром картина иная: траектории отдельных философов чередуются, между ними наблюдаются сдвиги по времени, и это отражает корректную синхронизацию доступа к вилкам. Таким образом, график показывает не только сам факт отсутствия тупика, но и то, что механизм арбитра обеспечивает продолжающееся обслуживание всех участников системы.

4.5 Параметрический сценарий

4.5.1 Полный код scripts/dining_philosophers_param_literate.jl

```
# # Параметрическое исследование
#
# Этот literate-скрипт добавляет дополнительное вычисление
# для набора параметров. Изучается, как число философов
# влияет на вероятность тупика
# в классической сети и в сети с арбитром.
#
# ## Подключение библиотек

using DrWatson
@quickactivate "lab_05_models"

ENV["GKSwstype"] = "100"

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers
using CSV
using DataFrames
using Plots

mkpath(datadir())
mkpath(plotsdir())

# ## Набор параметров
#
# Рассматриваются значения `N = 3:7`.
```

```julia
# Для каждого значения выполняется серия
# стохастических прогонов с разными `seed`.

philosopher_counts = 3:7
seeds = 1:16
tmax = 60.0

raw_scan = parameter_scan(philosopher_counts; seeds = seeds, tmax = tmax)
summary_scan = summarize_parameter_scan(raw_scan)

raw_scan
summary_scan

#jl CSV.write(datadir("parameter_scan_raw.csv"), raw_scan)
#jl CSV.write(datadir("parameter_scan_summary.csv"), summary_scan)

# ## Визуализация параметрического анализа
#
# На верхней панели показана вероятность deadlock,
# на нижней — среднее время его наступления.

parameter_plot = plot_parameter_summary(summary_scan)
parameter_plot

#jl savefig(parameter_plot, plotsdir("parameter_summary.png"))

# ## Интерпретация
#
# 1. Для классической сети вероятность deadlock
#    быстро растёт с увеличением `N`.
```

```
# 2. В серии экспериментов с арбитром тупики не наблюдаются.
# 3. Среднее время наступления deadlock
#    в классической сети остаётся конечным,
#    что подтверждает необходимость механизма синхронизации.
```

4.5.2 Запуск параметрического clean-скрипта

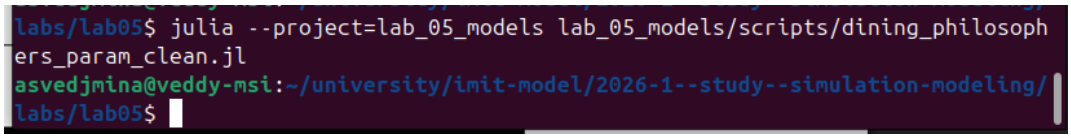


Рисунок 4.11: Запуск `dining_philosophers_param_clean.jl`

Во втором сценарии исследовалось влияние числа философов `N = 3:7` на вероятность взаимной блокировки. Для каждого значения `N` выполнялось `16` стохастических прогонов для классической сети и сети с арбитром.

4.5.3 Сводная таблица параметрического анализа

Таблица 4.2: Результаты параметрического исследования

Модель	N	Вероятность deadlock	Среднее время deadlock	Среднее число философов в Eat в конце
Классическая	3	1.0	2.3200	0.0
С арбитром	3	0.0	—	0.5
Классическая	4	1.0	4.6227	0.0
С арбитром	4	0.0	—	0.375
Классическая	5	1.0	8.9553	0.0
С арбитром	5	0.0	—	0.625
Классическая	6	1.0	4.3216	0.0

Модель	N	Вероятность deadlock	Среднее время deadlock	Среднее число философов в Eat в конце
С арбитром	6	0.0	—	0.5
Классическая	7	1.0	5.9987	0.0
С арбитром	7	0.0	—	0.625

4.5.4 График параметрического анализа

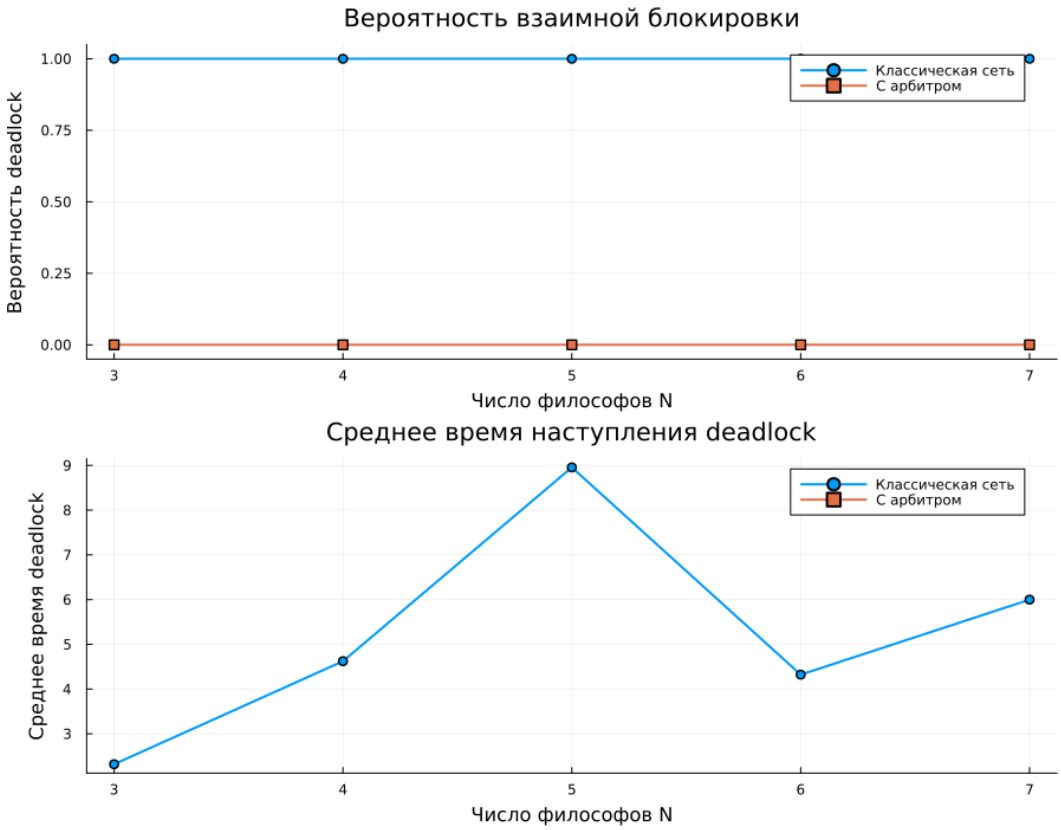


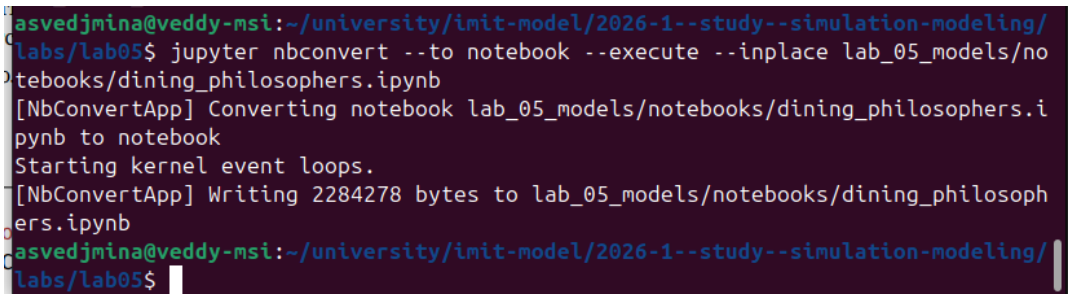
Рисунок 4.12: Вероятность и среднее время deadlock при изменении числа философов

Во всех сериях для классической сети вероятность `deadlock` оказалась равной `1.0`, а в сети с арбитром тупики не наблюдались ни разу.

На верхней панели параметрического графика кривая классической сети всё время остаётся на уровне `1.0`, то есть при каждом исследованном значении `N` взаимная блокировка возникала во всех шестнадцати прогонах. Кривая сети с арбитром, напротив, лежит на уровне `0.0`, что означает полное отсутствие тупиков в проведённой серии. Нижняя панель показывает, что среднее время наступления `deadlock` для классической сети остаётся конечным и зависит от числа философов: тупик может возникать как очень быстро, так и после более длительной переходной фазы. Тем не менее во всех случаях классическая схема неизбежно приходит к блокировке, тогда как арбитр качественно меняет режим работы сети.

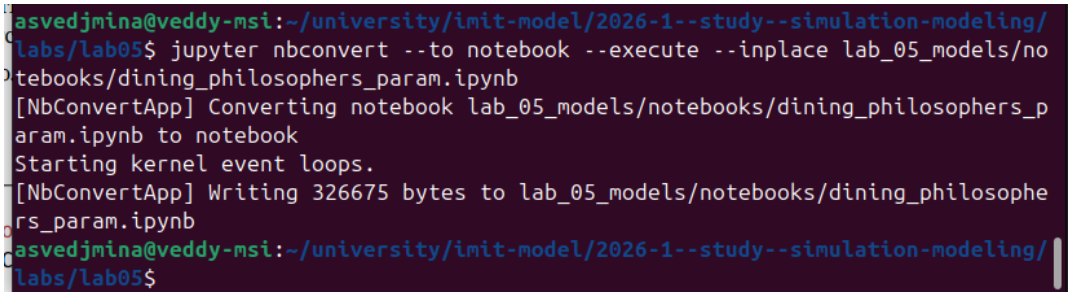
4.6 Выполнение Jupyter notebook

После генерации были выполнены оба notebook. Скриншоты консольных запусков приведены на рис. 4.13 и рис. 4.14.



```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$ jupyter nbconvert --to notebook --execute --inplace lab_05_models/no
tebooks/dining_philosophers.ipynb
[NbConvertApp] Converting notebook lab_05_models/notebooks/dining_philosophers.i
pynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 2284278 bytes to lab_05_models/notebooks/dining_philosoph
ers.ipynb
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$
```

Рисунок 4.13: Выполнение `dining_philosophers.ipynb`



```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$ jupyter nbconvert --to notebook --execute --inplace lab_05_models/no
tebooks/dining_philosophers_param.ipynb
[NbConvertApp] Converting notebook lab_05_models/notebooks/dining_philosophers_p
aram.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 326675 bytes to lab_05_models/notebooks/dining_philosophe
rs_param.ipynb
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab05$
```

Рисунок 4.14: Выполнение `dining_philosophers_param.ipynb`

Тем самым выполнено требование лабораторной работы о запуске кода из Jupyter notebook для базового и параметрического сценариев.

4.7 Автоматические тесты

Полный код проверки корректности вынесен в файл `test/runtests.jl`.

4.7.1 Полный код `test/runtests.jl`

```
using DataFrames
using DrWatson
using Random
using Test

@quickactivate "lab_05_models"

include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers

@testset "lab_05_models tests" begin
    net_classic, u0_classic, _ = build_classical_network(5)
    @test length(u0_classic) == 20
    @test sum(u0_classic[1:5]) == 5
    @test sum(u0_classic[16:20]) == 5
    @test !detect_deadlock(net_classic, u0_classic)
    @test length(enabled_transitions(net_classic, u0_classic)) == 5

    dead_marking = zeros(Float64, 20)
    dead_marking[6:10] .= 1.0
    @test detect_deadlock(net_classic, dead_marking)
```

```julia
    net_arbiter, u0_arbiter, _ = build_arbiter_network(5)
    @test u0_arbiter[end] == 4.0
    @test !detect_deadlock(net_arbiter, u0_arbiter)

    df = simulate_stochastic(net_classic, u0_classic, 5.0; rng = MersenneTwister(42))
    @test nrow(df) >= 2
    @test df.time[1] == 0.0

    raw = parameter_scan(3:4; seeds = 1:2, tmax = 10.0)
    summary = summarize_parameter_scan(raw)
    @test nrow(raw) == 8
    expected_names = [
        :model,
        :philosophers,
        :deadlock_probability,
        :mean_deadlock_time,
    ]
    @test all(name in Symbol.(names(summary)) for name in expected_names)
end
```

4.7.2 Запуск тестов

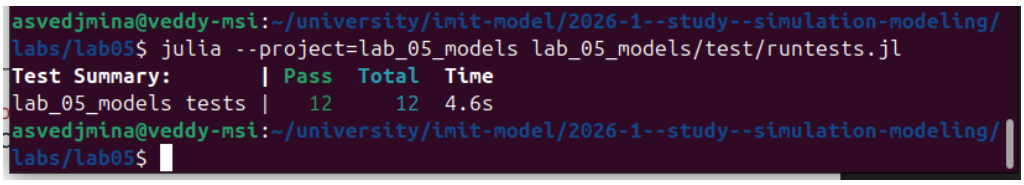


Рисунок 4.15: Результат запуска `runtests.jl`

Тесты подтвердили:

- корректность начальной маркировки;

- обнаружение искусственно заданного `deadlock`;
- отсутствие тупика в начальном состоянии сети с арбитром;
- корректность работы стохастической симуляции;
- корректность формирования таблиц параметрического анализа.

Всего успешно пройдено 12 тестов.

5 Выводы

В лабораторной работе была реализована воспроизводимая модель задачи обедающих философов в аппарате сетей Петри. Классическая сеть устойчиво приводит к взаимной блокировке, что подтверждается базовым стохастическим экспериментом и параметрической серией для `N = 3:7`.

Модификация с арбитром устраняет тупик во всех проведённых прогонах, сохраняет активность переходов и ненулевой поток в состояниях `Eat_i`. Дополнительно был организован literate-пайплайн, из которого автоматически получаются clean-скрипты, notebook и Quarto-документы; оба notebook были выполнены, а корректность кода подтверждена автоматическими тестами.

Список литературы

1. *Datseris G., contributors.* DrWatson.jl: the perfect sidekick to your scientific inquiries. — 2024. — URL: https://github.com/JuliaDynamics/DrWatson.jl (дата обр. 17.04.2026).
2. *Dijkstra E. W.* Hierarchical Ordering of Sequential Processes. — 1971. — EWD310.
3. *Gillespie D. T.* Exact Stochastic Simulation of Coupled Chemical Reactions // The Journal of Physical Chemistry. — 1977. — Т. 81, № 25. — С. 2340—2361. — DOI: 10.1021/j100540a008.
4. *Knuth D. E.* Literate Programming // The Computer Journal. — 1984. — Т. 27, № 2. — С. 97—111. — DOI: 10.1093/comjnl/27.2.97.
5. *Murata T.* Petri Nets: Properties, Analysis and Applications // Proceedings of the IEEE. — 1989. — Т. 77, № 4. — С. 541—580. — DOI: 10.1109/5.24143.
6. *Petri C. A.* Kommunikation mit Automaten : дис. … канд. / Petri Carl Adam. — Universität Hamburg, 1962.
7. *Plas F. van der, contributors.* Literate.jl. — 2025. — URL: https://github.com/fredrikekre/Literate.jl (дата обр. 17.04.2026).